

Enterprise Planning Platform

API Protocol & Integration Specification

Version 1.0 · Prepared for UI/Frontend Engineering

This specification documents all backend API endpoints, authentication mechanisms, JSON schemas, request/response payload examples, and integration workflows for the UI/Frontend engineering team

1. Global Specifications

Base URL: `http://localhost:8000`

Default Headers:

- Content-Type: `application/json`
- Accept: `application/json`

Authentication: Bearer Token in standard HTTP header

```
Authorization: Bearer <access_token>
```

2. Common Response Structures

2.1 Standard Error Response (HTTP 4xx / 5xx)

```
{
  "detail": "Human-readable error description"
}
```

2.2 Standard Action Response (OTPVerifyResponse)

```
{
  "success": true,
  "message": "Operation completed successfully.",
  "status": "ACTIVE"
}
```

3. Complete Endpoint Catalog

3.1 Authentication & Self-Service (/api/v1/auth)

1. Register / Signup

Endpoint: `POST /api/v1/auth/users`

Auth Required: No

Description: Initiates user registration and triggers an OTP to the user's email.

Request Body:

```
{
  "first_name": "John",
  "last_name": "Doe",
  "username": "johndoe",
  "email": "johndoe@example.com",
  "password": "StrongPassword@123",
  "role": "USER",
  "segment": [
    "Derma",
    "Cardio"
  ],
  "channel": [
    "Retail",
    "Hospital"
  ],
  "brand": [
    "BrandA",
    "BrandB"
  ]
}
```

Response (HTTP 201 Created):

```
{
  "success": true,
  "message": "Registration OTP sent to your email.",
  "status": "PENDING_OTP"
}
```

2. Verify OTP (Registration / Login)

Endpoint: POST /api/v1/auth/verify-otp or POST /api/v1/auth/otp/verify

Auth Required: No

Description: Verifies 6-digit OTP for signup or login challenges.

Request Body:

⚠ purpose must be one of: "REGISTRATION", "LOGIN", "PASSWORD_RESET", "PASSWORD_CHANGE".

```
{
  "email": "johndoe@example.com",
  "purpose": "LOGIN",
  "otp": "123456"
}
```

Response for Login OTP (HTTP 200 OK):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "expires_in": 900,
  "refresh_token": "def456...",
  "refresh_expires_in": 604800
}
```

Response for Registration OTP (HTTP 200 OK):

```
{
  "id": "2",
  "first_name": "John",
  "last_name": "Doe",
  "username": "johndoe",
  "email": "johndoe@example.com",
  "role": "USER",
  "status": "PENDING_APPROVAL",
  "created_at": "2026-07-21T10:00:00Z"
}
```

3. Resend OTP

Endpoint: POST /api/v1/auth/resend-otp, POST /api/v1/auth/send-otp, POST /api/v1/auth/otp

Auth Required: No

Request Body:

```
{
  "email": "johndoe@example.com",
  "purpose": "LOGIN"
}
```

Response (HTTP 200 OK):

```
{
  "success": true,
  "message": "OTP sent successfully.",
  "status": "ACTIVE"
}
```

4. Login (Password Step)

Endpoint: POST /api/v1/auth/login

Auth Required: No

Description: Validates credentials and returns an OTP challenge.

Request Body:

```
{
  "identifier": "johndoe@example.com",
  "password": "StrongPassword@123"
}
```

Response (HTTP 200 OK):

```
{
  "challenge_id": "a1b2c3d4-5678-90ab-cdef-1234567890ab",
  "purpose": "LOGIN",
  "email": "johndoe@example.com",
  "expires_at": "2026-07-21T11:15:00Z",
  "resend_available_at": "2026-07-21T11:06:00Z"
}
```

5. Refresh Access Token

Endpoint: POST /api/v1/auth/refresh-token

Auth Required: No

Request Body:

```
{
  "refresh_token": "def456..."
}
```

Response (HTTP 200 OK):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "expires_in": 900,
  "refresh_token": "new_refresh_token...",
  "refresh_expires_in": 604800
}
```

6. Request OTP for Password Change

Endpoint: POST /api/v1/auth/change-password/request-otp

Auth Required: Yes (Authorization: Bearer <access_token>)

Description: Sends a 6-digit verification OTP to the logged-in user's email.

Request Body: None

Response (HTTP 200 OK):

```
{
  "success": true,
  "message": "Password change OTP sent to your email.",
  "status": "ACTIVE"
}
```

7. Change Password (Authenticated)

Endpoint: POST /api/v1/auth/change-password

Auth Required: Yes (Authorization: Bearer <access_token>)

Description: Validates current password, OTP, and updates password hash.

Request Body:

```
{
  "current_password": "OldPassword@123",
  "new_password": "NewStrongPassword@123",
  "otp": "123456"
}
```

Response (HTTP 200 OK):

```
{
  "success": true,
  "message": "Password changed successfully.",
  "status": "ACTIVE"
}
```

8. Forgot Password (Initiate)

Endpoint: POST /api/v1/auth/forgot-password

Auth Required: No

Request Body:

```
{
  "email": "johndoe@example.com"
}
```

Response (HTTP 200 OK):

```
{
  "success": true,
  "message": "Password reset OTP sent to your email.",
  "status": "ACTIVE"
}
```

9. Reset Password (Unauthenticated OTP Reset)

Endpoint: POST /api/v1/auth/reset-password

Auth Required: No

Request Body:

```
{
  "email": "johndoe@example.com",
  "otp": "123456",
  "new_password": "NewStrongPassword@123"
}
```

Response (HTTP 200 OK):

```
{
  "success": true,
  "message": "Password reset successfully.",
  "status": "ACTIVE"
}
```

10. Get Logged-In User Profile

Endpoint: GET /api/v1/auth/users/me

Auth Required: Yes (Authorization: Bearer <access_token>)

Request Body: None

Response (HTTP 200 OK):

```
{
  "id": "1",
  "first_name": "System",
  "last_name": "Administrator",
  "username": "admin",
  "email": "admin@example.com",
  "role": "ADMIN",
  "roles": [
    "Administrator"
  ],
  "status": "ACTIVE",
  "segment": [
    "Derma"
  ],
  "channel": [
    "Retail"
  ],
  "brand": [
    "BrandA"
  ],
  "permissions": [
    "READ",
    "WRITE",
    "ADMIN"
  ],
  "created_at": "2026-07-20T23:51:32Z",
  "updated_at": "2026-07-21T00:00:34Z"
}
```

11. Logout

Endpoint: POST /api/v1/auth/logout

Auth Required: Yes (Authorization: Bearer <access_token>)

Request Body: None

Response (HTTP 200 OK):

```
{
  "success": true,
  "message": "Logged out successfully.",
  "data": null
}
```

3.2 Operational User Management APIs (/api/users)

1. List Users

Endpoint: GET /api/users

Auth Required: Yes (Authorization: Bearer <access_token>)

Allowed Roles: ADMIN, PLANNER, REVIEWER, FINANCE, LEADERSHIP

Response (HTTP 200 OK):

```
[
  {
    "id": "1",
    "first_name": "System",
    "last_name": "Administrator",
    "username": "admin",
    "email": "admin@example.com",
    "role": "ADMIN",
    "roles": [
      "Administrator"
    ],
    "status": "ACTIVE",
    "segment": [
      "Derma"
    ],
    "channel": [
      "Retail"
    ],
    "brand": [
      "BrandA"
    ],
    "permissions": [
      "ADMIN"
    ],
    "created_at": "2026-07-20T23:51:32Z",
    "updated_at": "2026-07-21T00:00:34Z"
  }
]
```

2. Create User Directly

Endpoint: POST /api/users

Auth Required: Yes (Authorization: Bearer <access_token>)

Request Body:

```
{
  "first_name": "Jane",
  "last_name": "Smith",
  "username": "janesmith",
  "email": "janesmith@example.com",
  "password": "StrongPassword@123",
  "role": "PLANNER",
  "segment": [
    "Derma"
  ],
  "channel": [
    "Hospital"
  ],
  "brand": [
    "BrandB"
  ]
}
```

Response (HTTP 201 Created): UserResponse object.

3. Get User By ID

Endpoint: GET /api/users/{user_id}

Auth Required: Yes (Authorization: Bearer <access_token>)

Response (HTTP 200 OK): UserResponse object.

4. Update User

Endpoint: PUT /api/users/{user_id}

Auth Required: Yes (Authorization: Bearer <access_token>)

Request Body:

```
{
  "first_name": "Jane",
  "last_name": "Doe-Smith",
  "role": "REVIEWER",
  "segment": [
    "Cardio"
  ],
  "channel": [
    "Retail"
  ],
  "brand": [
    "BrandA"
  ]
}
```

Response (HTTP 200 OK): Updated UserResponse object.

5. Delete User

Endpoint: DELETE /api/users/{user_id}

Auth Required: Yes (Admin token)

Response (HTTP 200 OK):

```
{
  "success": true,
  "message": "User '2' deleted successfully."
}
```

6. Bulk Download Template

Endpoint: GET /api/users/bulk/template

Auth Required: Yes

Response: Binary Excel file (.xlsx) stream (Content-Disposition: attachment; filename="bulk_users_template.xlsx").

7. Bulk Upload Users

Endpoint: POST /api/users/bulk/upload

Auth Required: Yes

Content-Type: multipart/form-data

Form Data: file (Excel spreadsheet .xlsx)

Response (HTTP 200 OK):

```
{
  "job_id": "e8a10a12-882e-4091-bf18-971a0670868f",
  "status": "PROCESSING",
  "total_rows": 150,
  "processed_rows": 0,
  "successful_rows": 0,
  "failed_rows": 0,
  "created_at": "2026-07-21T11:50:00Z",
  "completed_at": null
}
```

8. Check Bulk Upload Status & Errors

Get Job Status: GET /api/users/bulk/status/{job_id}

Get Job Errors: GET /api/users/bulk/errors/{job_id}

Errors Response (HTTP 200 OK):

```
{
  "job_id": "e8a10a12-882e-4091-bf18-971a0670868f",
  "status": "COMPLETED_WITH_ERRORS",
  "errors": [
    {
      "row_number": 4,
      "field": "email",
      "message": "Email address already registered.",
      "raw_value": "existing@example.com"
    }
  ]
}
```

3.3 Master Reference Data (/api)

Method	Path	Response
GET	/api/segments	{ "items": ["Derma", "Cardio", "Gastro"] }
GET	/api/channels	{ "items": ["Retail", "Hospital", "Institution"] }
GET	/api/brands	{ "items": ["BrandA", "BrandB", "BrandC"] }
GET	/api/roles	{ "items": [...6 roles] }

4. Key Workflows for UI Developers

[1. LOGIN FLOW]

```
User submits email + pass -> POST /api/v1/auth/login
<- Receives OTP challenge (purpose="LOGIN")
User enters 6-digit OTP -> POST /api/v1/auth/verify-otp
<- Receives access_token & refresh_token
Store access_token in Memory/Header & refresh_token in HTTP-only storage.
```

[2. CHANGE PASSWORD FLOW]

```
User enters current & new pass -> Clicks "Send OTP"
Frontend calls -> POST /api/v1/auth/change-password/request-otp (Bearer Token)
<- "Password change OTP sent to your email."
User enters OTP & submits -> POST /api/v1/auth/change-password (Bearer Token)
<- "Password changed successfully."
```

[3. EXPIRED TOKEN REFRESH FLOW]

```
API request returns 401 Unauthorized ->
Frontend calls -> POST /api/v1/auth/refresh-token with body { "refresh_token": "..."}
<- Receives new access_token & refresh_token
Retry failed API call with new access_token.
```